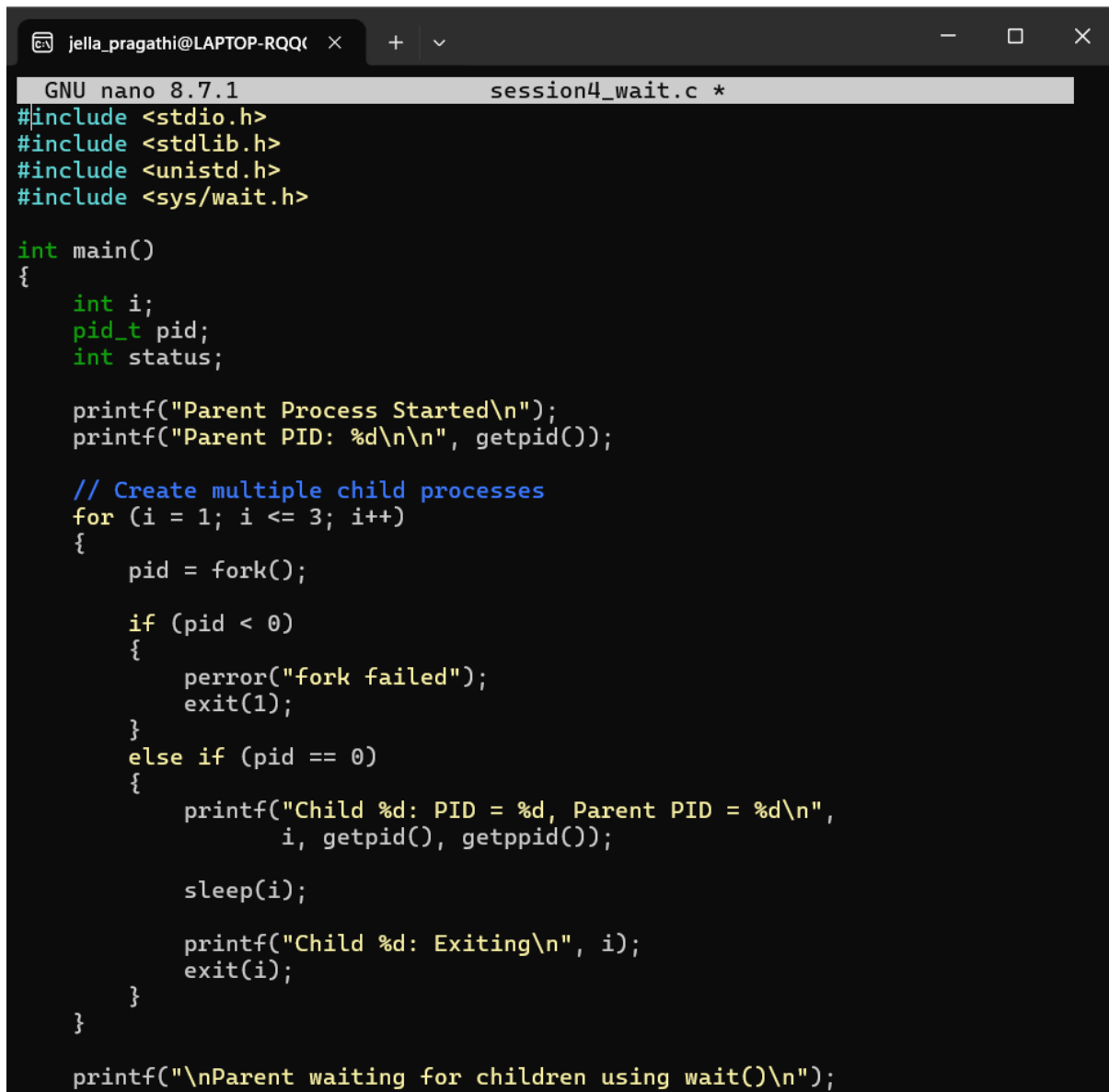


Practical week 4

Session 4

Task 1:

A screenshot of a terminal window with a dark background. The window title bar shows 'jella_pragathi@LAPTOP-RQQ...' and standard window controls. The terminal shows the GNU nano 8.7.1 editor editing a file named 'session4_wait.c'. The code is a C program that demonstrates process creation and waiting. It includes headers for stdio, stdlib, unistd, and sys/wait. The main function starts by printing 'Parent Process Started' and the parent's PID. It then enters a loop to create three child processes using fork(). For each child, it checks if the fork failed (pid < 0), prints an error and exits if so. If successful (pid == 0), it prints the child's PID, the parent's PID, sleeps for a duration 'i', prints 'Child Exiting', and exits. Finally, the parent process calls wait() to wait for all children to finish before printing a final message.

```
GNU nano 8.7.1 session4_wait.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    int i;
    pid_t pid;
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n\n", getpid());

    // Create multiple child processes
    for (i = 1; i <= 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }
        else if (pid == 0)
        {
            printf("Child %d: PID = %d, Parent PID = %d\n",
                   i, getpid(), getppid());

            sleep(i);

            printf("Child %d: Exiting\n", i);
            exit(i);
        }
    }

    printf("\nParent waiting for children using wait()\n");
}
```

```

// wait() waits for any one child
pid = wait(&status);

if (pid > 0)
{
    printf("wait(): Child with PID %d completed\n", pid);

    if (WIFEXITED(status))
        printf("Child exit status: %d\n", WEXITSTATUS(status));
}

printf("\nParent waiting for remaining children using waitpid()\n");

// waitpid() waits for a specific child
for (i = 0; i < 2; i++)
{
    pid = waitpid(-1, &status, 0);

    if (pid > 0)
    {
        printf("waitpid(): Child with PID %d completed\n", pid);

        if (WIFEXITED(status))
            printf("Child exit status: %d\n", WEXITSTATUS(status));
    }
}

printf("\nAll child processes completed.\n");
printf("Parent Process Exiting\n");

return 0;
}

```

```
jella_pragathi@LAPTOP-RQQGT555:~$ cd ~/OSSP
ls
LICENSE  README.md  bin  include  scripts  tests
Makefile assets  docs obj      src
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ nano session4_wait.c
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ gcc session4_wait.c -o session4_wait
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ ./session4_wait
Parent Process Started
Parent PID: 586

Child 1: PID = 587, Parent PID = 586
Child 2: PID = 588, Parent PID = 586

Child 3: PID = 589, Parent PID = 586
Parent waiting for children using wait()
Child 1: Exiting
wait(): Child with PID 587 completed
Child exit status: 1

Parent waiting for remaining children using waitpid()
Child 2: Exiting
waitpid(): Child with PID 588 completed
Child exit status: 2
Child 3: Exiting
waitpid(): Child with PID 589 completed
Child exit status: 3

All child processes completed.
Parent Process Exiting
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$
```

Task 2:

```
jella_pragathi@LAPTOP-RQQG555: ~/OSSP
GNU nano 8.7.1 zombie.c *
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;

    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        printf("Child process exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent sleeping for 20 seconds...\n");
        printf("Child PID: %d\n", pid);

        sleep(60);

        printf("Parent exiting...\n");
    }

    return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
jella_pragathi@LAPTOP-RQQG555:~/OSSP$ cd ~/OSSP
jella_pragathi@LAPTOP-RQQG555:~/OSSP$ nano zombie.c
jella_pragathi@LAPTOP-RQQG555:~/OSSP$ gcc zombie.c -o zombie
jella_pragathi@LAPTOP-RQQG555:~/OSSP$ ./zombie
Parent PID: 963
Parent sleeping for 20 seconds...
Child PID: 964
Child PID: 964
Child process exiting...
Parent exiting...
```

```
jella_pragathi@LAPTOP-RQQG555:~$ ps -o pid,ppid,state,cmd -p 964
PID      PPID  S  CMD
964      963  Z  [zombie] <defunct>
jella_pragathi@LAPTOP-RQQG555:~$ |
```

```
GNU nano 8.7.1 zombie.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid;
    int status;

    printf("Parent PID: %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0)
    {
        printf("Child PID: %d\n", getpid());
        printf("Child process exiting...\n");
        exit(0);
    }
    else
    {
        printf("Parent waiting for child...\n");
        printf("Child PID: %d\n", pid);

        wait(&status);

        if (WIFEXITED(status))
        {
            printf("Child completed successfully.\n");
            printf("Child exit status: %d\n", WEXITSTATUS(status));
        }
    }
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify

```

        printf("Child completed successfully.\n");
        printf("Child exit status: %d\n", WEXITSTATUS(status));
    }

    printf("Zombie process eliminated.\n");
}

return 0;
}

```

```

jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ cd ~/OSSP
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ nano zombie.c
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ gcc zombie.c -o zombie
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$ ./zombie
Parent PID: 1073
Parent waiting for child...
Child PID: 1074
Child PID: 1074
Child process exiting...
Child completed successfully.
Child exit status: 0
Zombie process eliminated.
jella_pragathi@LAPTOP-RQQGT555:~/OSSP$

```